МИНИСТЕРСТВО ЦИФРОВЫХ ТЕХНОЛОГИЙ РЕСПУБЛИКИ УЗБЕКИСТАН
ТАШКЕНТСКИЙ УНИВЕРСИТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ ИМЕНИ МУХАММАДА АЛ-ХОРАЗМИЙ

Дисциплина: «Введение в программную инженерию»

ПРАКТИЧЕСКАЯ РАБОТА № 6

Тема 6. Модель проектирования системы: архитектура, компоненты и UML-диаграммы

Вариант № 8
«Разработка умного сайта для заказа услуг»

Выполнила: студентка группы М320-24 ДИР
Камбарова Хатича

Проверила: Латипова Нодира

Ташкент — 2026

1. Тема практической работы

Модель проектирования системы: архитектура, компоненты и UML-диаграммы.

Цель работы: закрепить теоретические знания; научиться строить модель проектирования программной системы: декомпозировать систему на подсистемы, выбирать и обосновывать архитектурный стиль, строить UML-диаграммы компонентов, проектных классов и развёртывания, оценивать связность и сцепление модулей.

Вариант № 8: «Разработка умного сайта для заказа услуг».

2. Краткое описание системы и перечень подсистем

«XizmatGo» — умный веб-сайт (SPA/PWA) для заказа бытовых услуг в г. Ташкенте. Клиент находит услугу, оформляет заказ с адресом на карте и предварительной ценой, система подбирает лучшего свободного мастера, клиент отслеживает и оплачивает заказ и оставляет отзыв. Модель проектирования построена на основе ПР-3 (первоначальная архитектура, 8 модулей сервера) и ПР-5 (15 вариантов использования UC-01…UC-13, диаграмма классов анализа, требования FR-01…FR-12 и NFR-01…NFR-08). По сравнению с ПР-3 модули объединены в 7 прикладных подсистем, а общая техническая функциональность вынесена в 5 инфраструктурных модулей.

Таблица 2.1 — Подсистемы и инфраструктурные модули системы «XizmatGo»

ID	Подсистема / модуль	Назначение	Варианты использования (ПР-5)	Предоставляемый интерфейс (основные операции)	Требуемые интерфейсы
SS-1	**Доступ и идентификация**	Регистрация и вход по SMS-коду, JWT-токены, роли, профили, верификация мастеров	UC-01, UC-10	**IAuth**: sendCode(phone), verifyCode(phone, code): Token, getCurrentUser(token), checkRole(user, role)	INotify, IRepository
SS-2	**Каталог и поиск**	Категории, услуги, тарифы, умный поиск с подсказками RU/UZ	UC-02	**ICatalog**: listCategories(), search(text, lang), getService(id): ServiceInfo	IRepository
SS-3	**Управление заказами**	Оформление заказа, расчёт цены, фото, статусы, принятие заказа мастером	UC-03, UC-03.1, UC-03.2, UC-05, UC-08, UC-09	**IOrders**: createOrder(dto), getOrder(id), acceptOrder(id, masterId), changeStatus(id, status), markPaid(id, paymentId)	ICatalog, IMatching, IFileStorage, IEventBus, IRepository
SS-4	**Умный подбор**	Ранжирование свободных мастеров, передача заказа следующему через 5 мин	UC-04	**IMatching**: rankMasters(request): List<MasterRank>, offerNext(orderId)	IGeo, IRepository
SS-5	**Платежи**	Онлайн-оплата Payme/Click/Uzum, комиссия, фискальные чеки, возвраты	UC-06	**IPayments**: createPayment(orderId, method), handleCallback(provider, data), refund(paymentId)	IOrders, IEventBus, IRepository
SS-6	**Качество и модерация**	Отзывы и рейтинг, проверка мастеров, жалобы	UC-07, UC-11, UC-12	**IQuality**: addReview(orderId, mark, text), getRating(masterId), verifyMaster(id, decision), resolveComplaint(id, decision)	IEventBus, IRepository
SS-7	**Аналитика**	Дашборд руководителя, отчёты, выгрузка в Excel	UC-13	**IAnalytics**: getDashboard(period), exportReport(period): File	IEventBus (подписка), IRepository
INF-1	Уведомления	Отправка SMS (Eskiz) и Telegram-сообщений по шаблонам RU/UZ	(для UC-01, 04, 05)	**INotify**: send(userId, template, params)	IEventBus
INF-2	Геосервис	Геокодирование адресов, расчёт расстояний, поиск в радиусе	(для UC-03, UC-04)	**IGeo**: geocode(text), distance(a, b), findInRadius(point, km)	Яндекс Карты API
INF-3	Файловое хранилище	Загрузка и выдача фото и документов	(для UC-03.2, UC-10)	**IFileStorage**: upload(file): key, getUrl(key)	MinIO (S3 API)
INF-4	Шина событий и очередь задач	Асинхронный обмен событиями между подсистемами, фоновые задачи	(все)	**IEventBus**: publish(event), subscribe(type, handler)	Redis, Celery
INF-5	Доступ к данным	Репозитории и ORM, транзакции, миграции	(все)	**IRepository**: save, findById, find(criteria)	PostgreSQL + PostGIS

3. Выбранный архитектурный стиль и его обоснование

Выбран стиль **«многоуровневый модульный монолит с событийным взаимодействием модулей»**. Система разделена на уровни: представление (SPA, админ-панель) → API-шлюз → прикладные подсистемы → инфраструктура → данные. Каждая подсистема — отдельный модуль со своим интерфейсом и своими таблицами. Синхронные вызовы идут только через интерфейсы, а побочные действия (уведомления, аналитика) выполняются асинхронно через события. Всё серверное приложение собирается и разворачивается как один Docker-образ.

Таблица 3.1 — Обоснование выбора по атрибутам качества FURPS

Атрибут FURPS	Ключевые требования (ПР-5)	Как обеспечивает выбранный стиль	Модульный монолит	Микро-сервисы	Классический монолит
F — Functionality (функциональность, безопасность)	FR-01…FR-12; NFR-04: HTTPS, RBAC, OWASP; NFR-05: данные в РУз	Единый API-шлюз проверяет JWT и роли для всех подсистем; транзакции заказа и оплаты выполняются внутри одной БД без распределённых транзакций	5	4	4
U — Usability (удобство)	NFR-06: первый заказ ≤ 3 мин, mobile-first; NFR-07: RU/UZ	SPA/PWA без перезагрузки страниц; один API с единым форматом ошибок и локализацией	5	5	4
R — Reliability (надёжность)	NFR-03: доступность 99,5 %; FR-06: передача заказа через 5 мин; сбои Eskiz/Payme	Меньше сетевых точек отказа, чем у микросервисов; очередь Celery с повторами изолирует сбои внешних систем; ежедневный бэкап БД	4	3	3
P — Performance (производительность)	NFR-01: API ≤ 500 мс; FR-05: подбор ≤ 3 с; NFR-02: 1 000 пользователей	Вызовы между модулями внутри процесса (без сетевых задержек); кэш Redis; геоиндекс PostGIS; горизонтальное масштабирование копий API за Nginx	4	4	4
S — Supportability (сопровождаемость)	NFR-08: OpenAPI, покрытие ≥ 70 %; требования будут меняться после пилота	Чёткие границы модулей и интерфейсы позволяют менять подсистему независимо; одна кодовая база и один CI — посильно команде из 5 человек; модуль можно позже выделить в сервис	5	3	2
+ Ограничения (бюджет, команда, сроки)	5 человек, 6 мес., ≈ 383 млн сум; хостинг в РУз	Два VPS вместо кластера Kubernetes; не нужны отдельные БД, мониторинг и DevOps-инженер для каждого сервиса	5	1	5
		Итого (из 30):	**28**	**20**	**22**

Сравнение с альтернативами. Микросервисная архитектура лучше масштабируется по отдельным сервисам. Но для команды из 5 человек она дорога и сложна: сетевые вызовы, распределённые транзакции «заказ — оплата», отдельное развёртывание и мониторинг каждого сервиса. **Классический монолит** прост и дёшев, но без границ модулей быстро превращается в «большой ком грязи», а это опасно, потому что требования будут меняться после пилота. Модульный монолит сочетает простоту развёртывания монолита с модифицируемостью, близкой к микросервисам, и набирает 28 баллов из 30.

4. Диаграмма компонентов и диаграмма проектных классов

4.1. Диаграмма компонентов

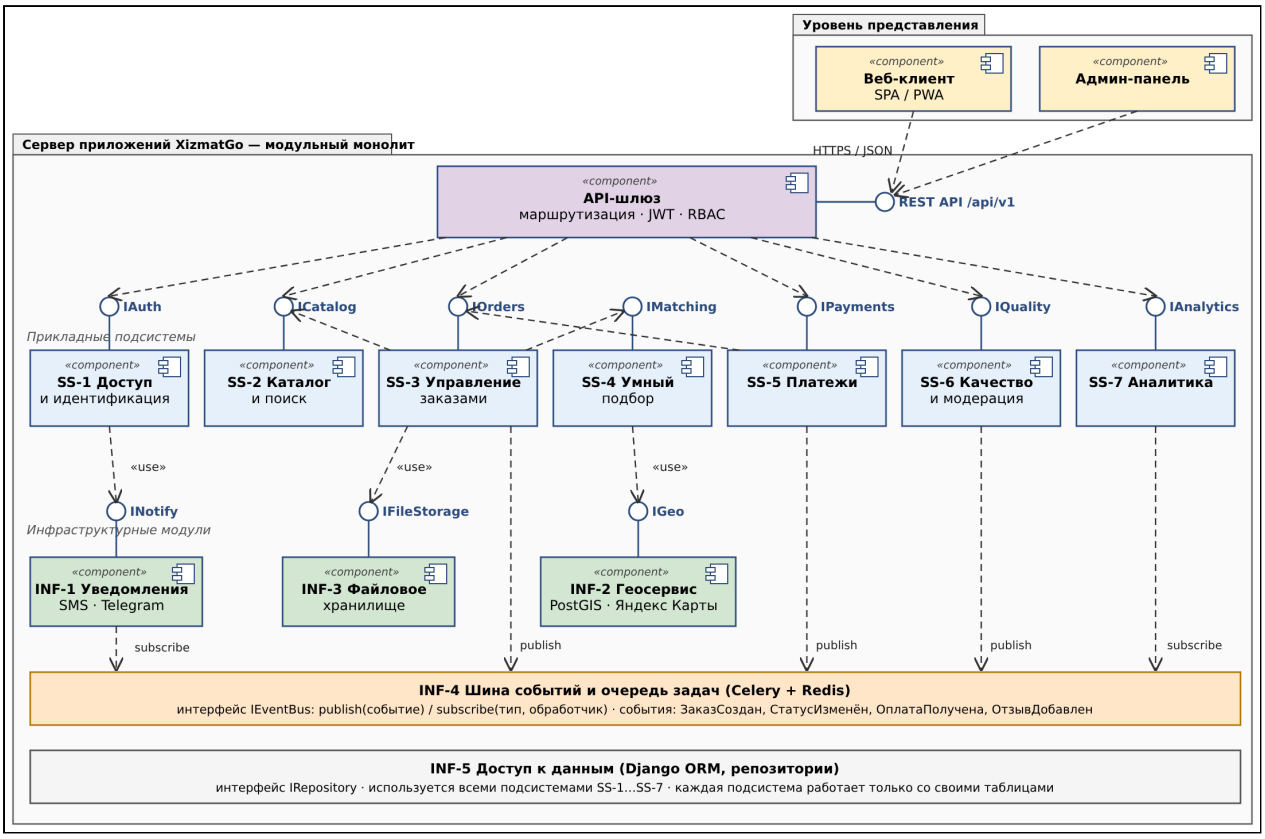


Рисунок 4.1 — UML-диаграмма компонентов системы «XizmatGo» (после рефакторинга, см. раздел 5)

Пояснения. Компоненты обозначены прямоугольниками со значком компонента. Предоставляемые интерфейсы показаны «леденцом» (кружок на линии), зависимости от требуемых интерфейсов — пунктирными стрелками. Веб-клиент и админ-панель обращаются только к интерфейсу REST API шлюза. Шлюз вызывает интерфейсы 6 подсистем; интерфейс IMatching внутренний и доступен только подсистеме заказов. Прямые синхронные зависимости между подсистемами сведены к трём: SS-3 → ICatalog, SS-3 → IMatching, SS-5 → IOrders. Уведомления и аналитика получают данные через подписку на события шины INF-4, поэтому остальные подсистемы о них не знают. Все подсистемы используют модуль доступа к данным INF-5, но каждая работает только со своими таблицами.

4.2. Диаграмма проектных классов подсистемы SS-3 «Управление заказами»

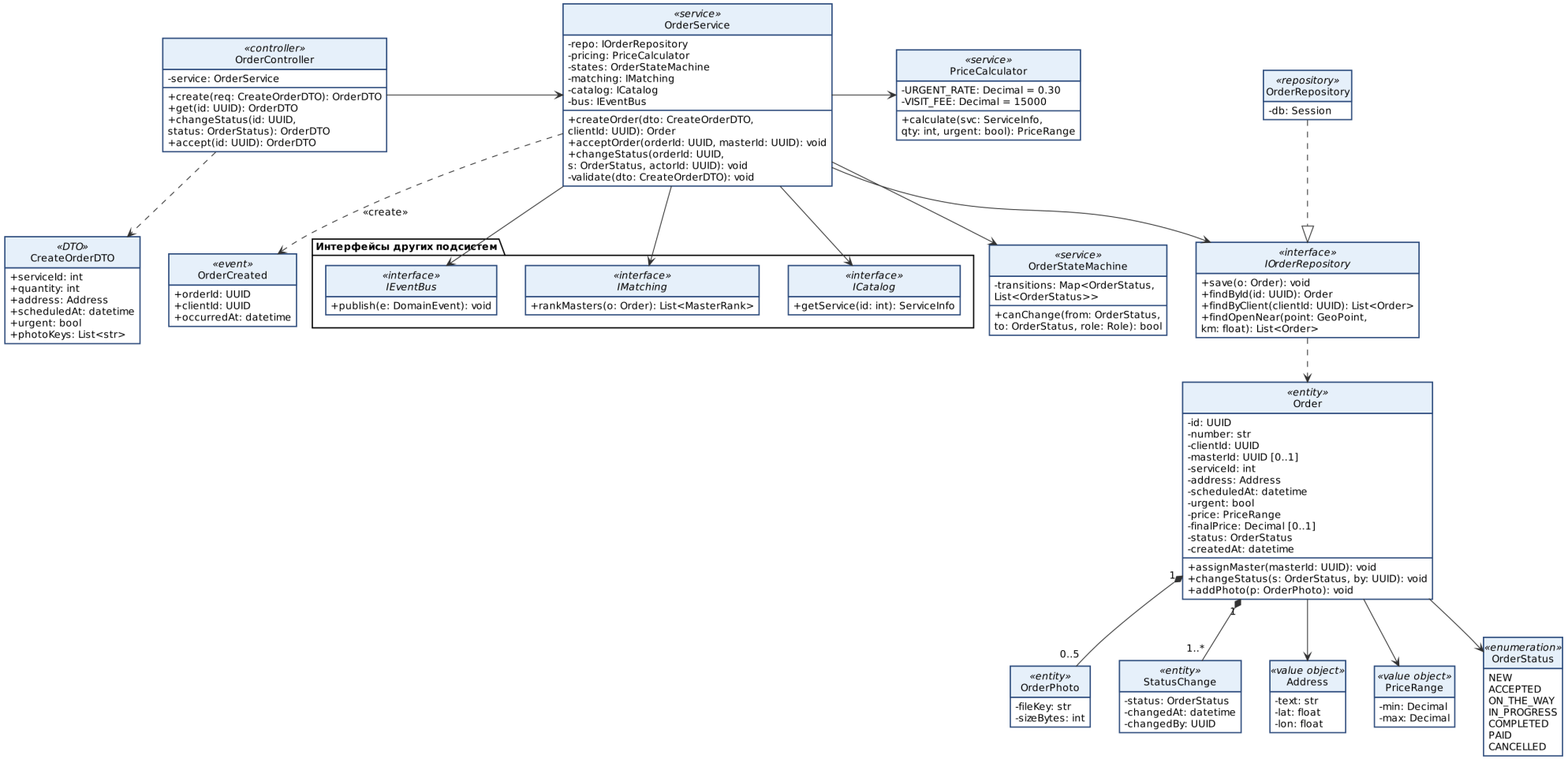


Рисунок 4.2 — Диаграмма проектных классов подсистемы SS-3 «Управление заказами»

Таблица 4.1 — Типы проектных классов подсистемы SS-3 и их происхождение из модели анализа (ПР-5)

Проектный класс	Тип (стереотип)	Ответственность	Класс анализа (ПР-5)
OrderController	«controller»	REST-эндпоинты /orders: приём запроса, проверка прав, преобразование DTO	ФормаЗаказа (boundary)
CreateOrderDTO	«DTO»	Структура входных данных запроса без логики	—
OrderService	«service»	Сценарии UC-03, UC-08, UC-09: создание, принятие, смена статуса; публикация событий	КонтроллерЗаказа (control)
PriceCalculator	«service»	Расчёт диапазона цены (база × объём + выезд + срочность)	СервисРасчётаЦены (control)
OrderStateMachine	«service»	Допустимые переходы статусов и права ролей	— (выделен при проектировании)
IOrderRepository / OrderRepository	«interface» / «repository»	Сохранение и поиск заказов; реализация на Django ORM (отношение реализации)	—
Order, OrderPhoto, StatusChange	«entity»	Хранимые данные заказа; Order владеет фото и историей статусов (композиция)	Заказ, ФотоЗаказа (entity)
Address, PriceRange	«value object»	Неизменяемые значения без собственной идентичности	атрибуты класса «Заказ»
OrderStatus	«enumeration»	Перечень статусов: NEW → ACCEPTED → ON_THE_WAY → IN_PROGRESS → COMPLETED → PAID; CANCELLED	атрибут «статус»
OrderCreated	«event»	Доменное событие для уведомлений и аналитики	—
IMatching, ICatalog, IEventBus	«interface»	Интерфейсы других подсистем, от которых зависит SS-3 (SS-4, SS-2, INF-4)	СервисПодбора (control), Услуга (entity)

Отношения на диаграмме: направленные ассоциации (сервис хранит ссылки на репозиторий, калькулятор и интерфейсы); зависимости «create» и использование DTO (пунктир); реализация интерфейса IOrderRepository классом OrderRepository; композиции Order ◆— OrderPhoto (0..5) и Order ◆— StatusChange (1..*). Сервис зависит от интерфейсов, а не от конкретных классов других подсистем (принцип инверсии зависимостей), поэтому его можно тестировать с заглушками.

5. Оценка связности и сцепления модулей, рефакторинг

Шкалы. Связность (cohesion) — от лучшей к худшей: функциональная (10), последовательная (9), коммуникационная (7), процедурная (5), временная (3), логическая (1), случайная (0). Сцепление (coupling) — от лучшего к худшему: по данным (1), по образцу (3), по управлению (4), внешнее (5), общее (7), по содержимому (9). Оценка выполнена для первоначальной архитектуры из ПР-3 («до»). На диаграмме 4.1 показан результат после рефакторинга («после»).

Таблица 5.1 — Связность модулей

Модуль	Связность «до»	Проблема	Предложение по рефакторингу	Связность «после»
SS-1 Доступ	Функциональная (10)	—	—	Функциональная (10)
SS-2 Каталог	Функциональная (10)	—	—	Функциональная (10)
SS-3 Заказы	Коммуникационная (7)	Модуль сам рассылал SMS и пересчитывал рейтинг мастера после завершения заказа — чужие обязанности над общими данными заказа	Рассылку и рейтинг вынести в подписчиков событий СтатусИзменён / ЗаказЗавершён; статусы — в OrderStateMachine	Функциональная (10)
SS-4 Подбор	Последовательная (9)	Внутри был геокод адреса — выход одного шага передаётся следующему	Геокодирование вынести в INF-2 (IGeo)	Функциональная (10)
SS-5 Платежи	Функциональная (10)	—	—	Функциональная (10)
SS-6 Качество	Логическая (1)	Объединены несвязанные функции «по теме качество»: отзывы, верификация мастеров, жалобы; выбор функции — по флагу типа	Разделить на два подмодуля внутри SS-6: «Отзывы и рейтинг» и «Модерация» (верификация + жалобы) с отдельными интерфейсами IReviews и IModeration	Функциональная (10) у каждого подмодуля
SS-7 Аналитика	Коммуникационная (7)	Работа над чужими таблицами разных модулей	Собственная витрина данных (read model), наполняемая событиями	Функциональная (10)
INF-1 Уведомления	Временная (3)	«Всё, что отправляется после смены статуса», собрано в одной функции	Обработчик на каждый тип события + единый шаблонизатор send(template)	Функциональная (10)

Таблица 5.2 — Сцепление между модулями

Связь	Сцепление «до»	Проблемное место	Рефакторинг	Сцепление «после»
SS-5 → SS-3	По содержимому (9)	Платежи напрямую меняли поле status в таблице заказов	Вызов IOrders.markPaid(id, paymentId) + событие ОплатаПолучена	По данным (1)

SS-7 → все	Общее (7)	Аналитика читала таблицы всех модулей; любое изменение схемы ломало отчёты	Подписка на события и собственная витрина данных	По данным через события (1)
SS-3 ↔ SS-4	По образцу (3) + цикл	Заказы передавали в подбор весь объект Order, а подбор вызывал SS-3, чтобы закрепить мастера, — циклическая зависимость	Передавать MatchRequest (услуга, координаты, время); подбор только возвращает рейтинг, мастера закрепляет SS-3	По данным (1), без цикла
SS-3 → INF-1	По управлению (4)	Вызов notify(order, channel="sms", urgent=True) — флаги управляют логикой модуля уведомлений	Публикация события СтатусИзменён; канал выбирает подписчик по настройкам пользователя	По данным через события (1)
SS-3 → SS-2	По образцу (3)	Получение всей модели Service из каталога, хотя нужны 3 поля	Возвращать ServiceInfo (id, базоваяЦена, единица)	По данным (1)
SS-1 → INF-1	По данным (1)	—	—	По данным (1)
Все → INF-5	Внешнее (5)	Зависимость от общей ORM и общей конфигурации БД — неизбежна	Доступ только через репозитории своего модуля; запрет межмодульных JOIN-ов, проверка в код-ревью	Внешнее (5), контролируемое

Итог. Средняя связность выросла с 7,1 до 10 баллов. Количество связей с сильным сцеплением (по содержимому, общее, по управлению) сократилось с 3 до 0, циклическая зависимость SS-3 ↔ SS-4 устранена. Остаётся только контролируемое внешнее сцепление с модулем доступа к данным.

6. Диаграмма развёртывания

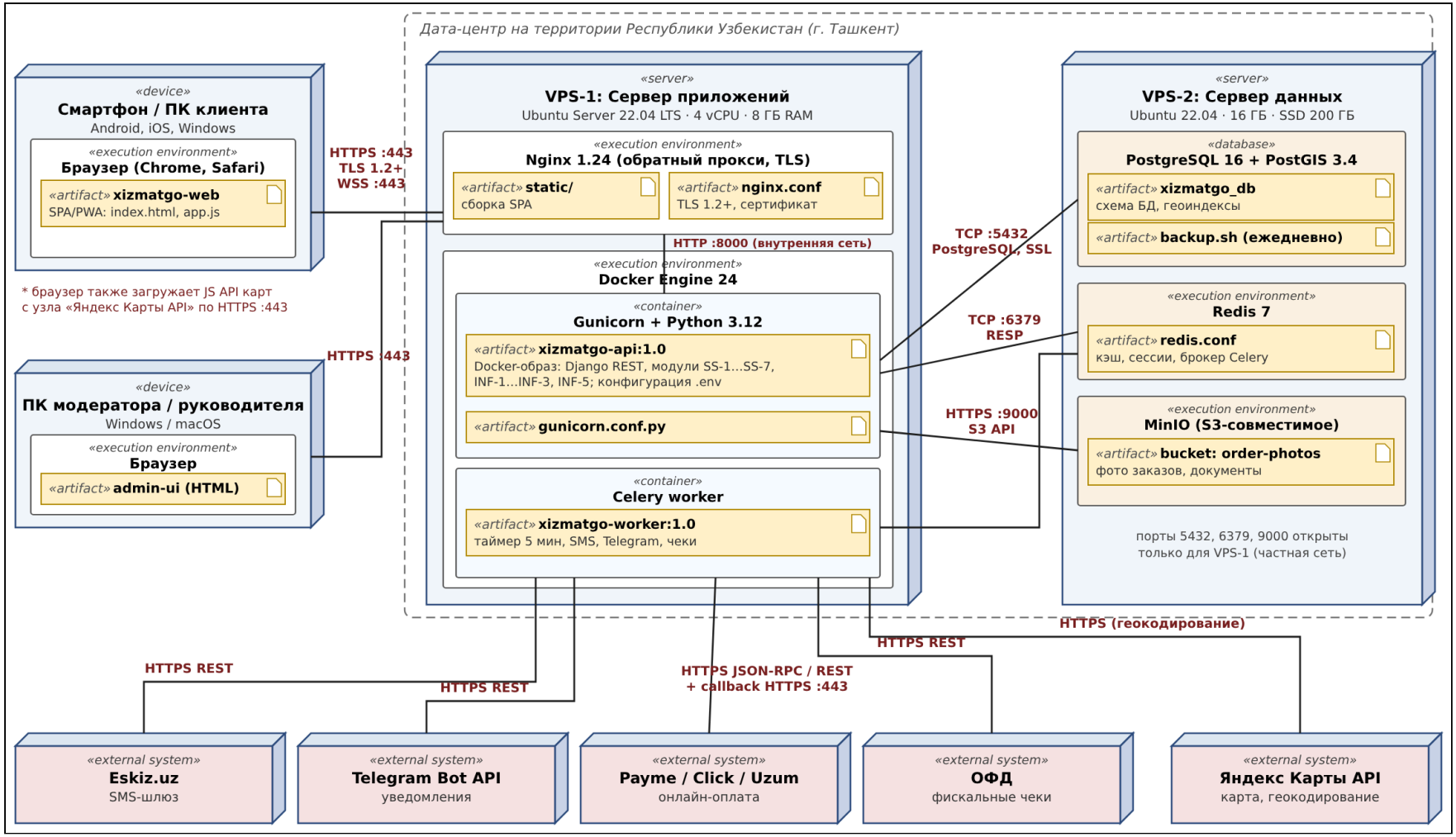


Рисунок 6.1 — UML-диаграмма развёртывания системы «XizmatGo» (дескрипторная форма)

Диаграмма построена в **дескрипторной форме**: показаны типы узлов (устройства, серверы, среды выполнения), размещённые на них артефакты и соединения с протоколами, а не конкретные экземпляры машин. Так, узел «Смартфон / ПК клиента» обозначает любое устройство клиента.

Таблица 6.1 — Соединения между узлами

Соединение	Протокол / порт	Назначение
Браузер клиента / модератора → Nginx	HTTPS :443 (TLS 1.2+), WSS :443	Загрузка SPA, вызовы REST API, обновление статусов в реальном времени
Nginx → Gunicorn (xizmatgo-api)	HTTP :8000, внутренняя сеть Docker	Проксирование запросов к API
xizmatgo-api → PostgreSQL	TCP :5432 (протокол PostgreSQL, SSL)	Хранение данных, геозапросы PostGIS
xizmatgo-api, worker → Redis	TCP :6379 (RESP)	Кэш, сессии, брокер очереди Celery
xizmatgo-api → MinIO	HTTPS :9000 (S3 API)	Загрузка и выдача фото заказов
worker → Eskiz.uz, Telegram, ОФД	HTTPS REST	SMS, уведомления, фискальные чеки
xizmatgo-api ↔ Payme / Click / Uzum	HTTPS JSON-RPC / REST; callback на :443	Создание платежа и получение подтверждения оплаты
xizmatgo-api, браузер → Яндекс Карты API	HTTPS :443	Геокодирование, расчёт расстояний, отображение карты

Оба сервера размещаются в дата-центре на территории Узбекистана (требование NFR-05). Порты сервера данных открыты только для сервера приложений в частной сети. Для роста нагрузки (NFR-02) можно добавить второй узел приложений за тем же Nginx: сервер API не хранит состояние, так как сессии лежат в Redis.

7. Вывод о проделанной работе

В практической работе по варианту № 8 «Разработка умного сайта для заказа услуг» построена модель проектирования системы «XizmatGo»:

1. На основе вариантов использования и модели анализа из ПР-5 система декомпозирована на 7 прикладных подсистем (SS-1…SS-7) и 5 инфраструктурных модулей (INF-1…INF-5). Для каждой указаны назначение, поддерживаемые варианты использования, предоставляемый и требуемые интерфейсы.
2. Выбран стиль «многоуровневый модульный монолит с событийным взаимодействием». Выбор обоснован требованиями и атрибутами FURPS и сравнением с микросервисами и классическим монолитом: 28, 20 и 22 балла из 30.

3. Построены UML-диаграмма компонентов с интерфейсами и зависимостями и диаграмма проектных классов подсистемы «Управление заказами». На ней 17 классов разных типов (контроллер, сервисы, репозиторий, сущности, объекты-значения, DTO, перечисление, событие, интерфейсы) с атрибутами, операциями и отношениями.
4. Оценены связность 8 модулей и сцепление 7 связей между ними. Найдено 5 проблемных мест: логическая связность модуля качества, сцепление по содержимому и общее сцепление, сцепление по управлению, циклическая зависимость. Предложен рефакторинг: события, интерфейсы, разделение модуля, витрина данных. После него средняя связность выросла с 7,1 до 10, а сильное сцепление устранено.
5. Построена диаграмма развёртывания в дескрипторной форме: 2 клиентских устройства, 2 сервера в дата-центре РУз, 5 внешних систем, артефакты и соединения с протоколами и портами.

Модель проектирования связывает требования (ПР-5) с конкретной структурой кода и инфраструктурой. Она служит основой для реализации и тестирования системы в следующих работах.